

Banking Chatbot — Project Explanation

Overview

This project, Part 6 of my Generative AI learning series, builds a domain-scoped banking chatbot named Eva using Streamlit's native chat interface combined with the OpenAI API. It fuses the frontend fundamentals from Part 5 with the persona and memory concepts from Part 4 into a single working conversational application.

What the Application Does

The app opens with a title and an assistant greeting, then maintains a persistent chat history using Streamlit's session state so that previous messages remain visible across the framework's full-script reruns. User input is captured through Streamlit's dedicated `chat_input` widget rather than a generic text box, and each message is displayed immediately in a chat bubble. Every user message, together with the full conversation so far, is sent to `gpt-4o-mini` alongside a system prompt that defines Eva's identity as a professional banker, restricts her scope to banking-related queries, and instructs her to politely decline anything outside that domain.

A notable detail examined in the accompanying README is that the assistant's own replies are not saved back into session state, since the line appending them to memory is commented out. Only the user's side of the conversation accumulates, meaning Eva loses track of what she previously said even as she retains full context of what the user has asked. This is documented as a deliberate learning point about the importance of testing multi-turn conversations rather than single exchanges.

Skills Demonstrated

- Streamlit's native chat UI: `st.chat_input` and `st.chat_message`
- Persistent state management with `st.session_state` across reruns
- Persona-driven system prompting for domain-scoped assistants
- Debugging multi-turn conversational memory issues

This project is the direct architectural precursor to my deployed LangChain-OpenAI chatbot web application.